

Báo cáo kỹ thuật: BudgetRAG và RLAIIF cho lựa chọn retrieval-context

Branch: feature/rlaif-retrieval-context-v0

Ngày 04 tháng 06 năm 2026

Mốc code hiện tại	Schema, rlaif-build, rlaif-label-answers, rlaif-label-contexts, rlaif-label-pairs, rlaif-reward -answer-labels, opt-in pairwise_tie_v1, rlaif-split, rlaif-train, rlaif-eval, family_smoothed_best_average, shrinkage_smoothed_best_average, linear_reward_model, smoothed_linear_selector, multi-seed selector sweep, action coverage diagnostics, answer/context/pairwise-label summaries, pairwise calibration diagnostics và Qwen KV estimates
Mốc schema	fdefb63 – feat(rlaif): add retrieval-context schemas
Mốc kế hoạch	c634bd3 – docs(plan): update rlaif retrieval-context roadmap
Trạng thái test gần nhất	205 passed

1 Tóm tắt

BudgetRAG hiện đã chuyển từ một hệ thống so sánh chiến thuật context-budget thủ công sang một nền tảng có thể tạo dữ liệu RLAIIF offline. Điểm thay đổi quan trọng là action không còn chỉ là “giữ bao nhiêu context”, mà là một **retrieval-context action**: chọn retrieval strategy, fusion strategy, top-k, context policy, budget, adaptive profile, selected context action và generator model.

Các commit gần nhất tạo nền cho Phase 1C.3:

- fdefb63: định nghĩa schema cho action, answer feedback, context feedback, scalar reward và preference.
- 78cf83d: thêm rlaif-build để đọc BudgetRAG output và ghi các dataset RLAIIF chuẩn hóa.
- Hardening hiện tại: chuẩn hóa AI judge provenance thành ai_judge và cho phép full-context/legacy action không có explicit budget.
- Answer judge hiện tại: thêm rlaif-label-answers với resume, progress logging, JSON repair và guardrail không biến invalid/missing label thành score 0.
- Context judge hiện tại: thêm rlaif-label-contexts để label sufficient context, selected/redundant/irrelevant chunks, missing evidence và context quality với cùng cơ chế dry-run/resume/JSON repair; thêm script summary và report template cho context labels.

- Pairwise judge hiện tại: thêm `rlaif-label-pairs` để judge trực tiếp action A/B từ `rlaif_preferences.jsonl`, nhưng không bắt buộc tin preference cũ; output có A/B/tie/ambiguous, quality/support/efficiency winner, quality regret, unsupported-claim risk và confidence.
- Reward/preference hiện tại: thêm `rlaif-reward` để ghi reward, preference và summary từ dataset RLAIIF đã chuẩn hóa, gồm optional `-answer-labels` để dùng AI-judge labels hợp lệ.
- Offline selector hiện tại: thêm `rlaif-split`, `rlaif-train` và `rlaif-eval` cho held-out query evaluation với fixed, cheapest, best-average, family_smoothed_best_average, shrinkage_smoothed_best_average, linear_reward_model, smoothed_linear_selector và oracle-logged baselines.
- Multi-seed selector hiện tại: thêm `scripts/run_rlaif_split_sweep.py` để chạy nhiều split seed và báo mean/std cho selector metrics.
- Action coverage hiện tại: thêm `scripts/inspect_rlaif_action_coverage.py` để phân tích exact signature sparsity và collapsed family coverage.
- AI-judge held-out hiện tại: full MiMo answer-label run trên Kaggle đã hoàn tất; reward được rebuild bằng `-answer-labels`, rerun split/train/eval và ghi report docs/reports/phase1d_rlaif_ai_judge_heldout_eval.md.
- Direct pairwise audit hiện tại: chạy MiMo trên 50 reward-derived preference pairs, ghi 50 valid JSON labels, 2 ambiguous timeouts, 41 A wins, 7 B wins và agreement 0.854 trên non-ambiguous decisions; report nằm ở docs/reports/phase1d_rlaif_pairwise_mimo50.md.
- Pairwise calibration diagnostics hiện tại: thêm `scripts/diagnose_rlaif_pairwise_calibration.py`; MiMo-50 diagnostics ghi 38 small quality/support delta pairs, 35 cheaper-wins-when-tied và 5 scalar-over-quality disagreements.
- Reward calibration candidate hiện tại: thêm opt-in `-reward-calibration pairwise_tie_v1`; default vẫn là none, scalar reward rows không đổi, chỉ preference construction dùng efficiency tie-break khi quality/support nằm trong tie band.

Đây vẫn là bước dữ liệu và đánh giá offline. Hệ thống **chưa** thay adaptive-heuristic bằng learned policy, **chưa** train contextual bandit thật, **chưa** có full context-level judge run, và **chưa** thực hiện KV pruning runtime. Ranh giới này quan trọng để tránh “fake RL”.

2 Bối cảnh nghiên cứu

Project nên được đặt tại giao điểm của sáu cụm nghiên cứu: RAG, evaluation, prompt/context compression, KV-cache compression, GraphRAG/retrieval strategy selection, và RLAIIF/contextual bandit.

RAG đặt nền cho việc kết hợp parametric memory với retrieved external memory trong generation [7]. BEIR cung cấp chuẩn đánh giá retrieval đa domain và cho thấy retriever không có một cấu hình tốt tuyệt đối trên mọi tập [13]. RAGAS bổ sung góc nhìn evaluation ở cấp RAG, đặc biệt là faithfulness, answer relevance và context relevance [4].

BudgetRAG khác các prompt compression chung như LLMLingua, LongLLMLingua và Selective Context [5, 6, 9] ở chỗ compression được coi là một **action trong bài toán retrieval-grounded decision**. Policy phải quyết định giữ bằng chứng nào dưới ràng buộc chất lượng, token, latency và estimated KV, thay vì chỉ nén prompt theo một mục tiêu cố định.

Các hướng KV-cache như H2O, StreamingLLM, KIVI, SnapKV và PyramidKV [15, 14, 11, 10, 2] can thiệp sâu hơn vào runtime/model cache. BudgetRAG hiện đi theo hướng ít xâm lấn hơn: giảm context trước

inference, sau đó dùng context-level RLAIIF labels làm bằng chứng cho bước evidence mask/KV retention proof of concept.

GraphRAG nhấn mạnh vai trò của graph-based text index cho các truy vấn global hoặc sensemaking [3]. Vì vậy action space của Phase 1D phải bao gồm retrieval strategy, không chỉ context budget.

RLAIIF và preference learning cung cấp cách biến feedback từ AI judge thành reward/preference [1, 12]. Với BudgetRAG, feedback này không nhằm alignment chatbot tổng quát, mà nhằm chấm **context sufficiency** và **grounded answer quality**. Vì mỗi query chỉ cần một lựa chọn retrieval-context ở một bước, offline contextual bandit là formulation đầu tiên hợp lý hơn PPO/GRPO [8].

3 Định vị so với related work

Research line	Trọng tâm	Vị trí của BudgetRAG
RAG	Retrieve external evidence cho generation	Thêm bài toán phân bổ context theo tài nguyên và chất lượng.
BEIR/RAGAS	Đánh giá retrieval/RAG	Mở rộng metric bằng token, latency, estimated KV và feedback provenance.
Prompt compression	Prompt/context compression	Biến compression thành query-level decision action trong RAG.
KV-cache methods	Runtime KV compression/pruning	Bắt đầu model-agnostic, sau đó dùng evidence labels cho KV retention.
RLAIIF/DPO	AI/preference feedback	Label context sufficiency và grounded answer quality.
Contextual bandit	Học one-step action policy từ feedback	Học retrieval-context action dưới reward có quality guardrail.

4 Trạng thái implementation hiện tại

4.1 Schema

Module `rag_bench.rlaif_schema` định nghĩa:

- `RetrievalContextAction`: state/action ở cấp query, bao gồm retriever, fusion, top-k, context policy, optional budget, adaptive profile, selected context policy/budget và generator model. Full-context hoặc legacy action thiếu explicit budget dùng `budget_chars=None`.
- `RlaifAnswerFeedback`: feedback cấp answer với provenance gold, ragas, ai_judge, mimo_judge cho backward compatibility, heuristic, missing.
- `RlaifContextFeedback`: feedback cấp context với sufficient, selected/redundant/irrelevant chunks, missing evidence, minimality score, evidence support score và context quality score.
- `RlaifReward`: reward tách quality/support/token/latency/KV/error/unsupported claim.
- `RlaifPreference`: preference type cho context-policy, retrieval-context và context-sufficiency.

Action id deterministic là một điểm đúng hướng: id gồm benchmark, query id, retrieval strategy, fusion strategy, top-k, context policy, budget, adaptive profile, selected context action và generator model, nhưng loại source run id để nhiều lần chạy cùng cấu hình có thể merge được.

4.2 Dataset builder

Command hiện có:

```
uv run rag-bench rlaif-build \
  --inputs benchmark_results/budgetrag/<matrix-run> \
  --output-dir benchmark_results/rlaif/<run-name>
```

Builder thực hiện:

- discover mọi `query_results.jsonl` trong input file hoặc directory;
- convert từng row thành normalized action qua `action_from_budgetrag_row`;
- giữ observation payload: answer text, retrieved records, context metrics, retrieval metrics, retrieval metadata, KV estimate, latency, token usage, generation error/retry/rate-limit metadata;
- tạo answer feedback theo thứ tự: gold EM/F1, RAGAS fields, AI judge fields, rồi missing;
- ghi summary với provenance counts, missing reason counts, generation error count và invalid rows.

Quan trọng: missing feedback không bị gán score 0. Generation skipped, generation error và missing answer được ghi bằng `missing_reason` riêng, trong đó generation error được đánh dấu `ambiguous=True`.

4.3 Answer judge labeler

Command hiện có:

```
uv run rag-bench rlaif-label-answers \
  --actions benchmark_results/rlaif/<run-name>/rlaif_actions.jsonl \
  --output benchmark_results/rlaif/<run-name>/rlaif_answer_labels_mimo.jsonl \
  --judge-provider mimo \
  --judge-model mimo-v2.5-pro \
  --resume \
  --json-retries 1 \
  --max-completion-tokens 4096
```

`rlaif-label-answers` chỉ judge từ question, answer và retrieved context đã log; prompt cấm browse và external knowledge. Output ghi incremental nên có thể resume sau khi kernel/local process bị ngắt. Nếu model trả invalid JSON, empty content, thiếu answer hoặc thiếu context, label được ghi là ambiguous với `quality_score=null`; không có path nào biến lỗi judge thành score 0.

Script `scripts/summarize_rlaif_labels.py` đọc `rlaif_answer_labels.jsonl` và báo label count, valid/invalid JSON, ambiguous labels, judge provider/model counts, score mean/std, unsupported-claim penalty và correlation với RAGAS answer relevancy khi có `rlaif_feedback.jsonl`. Template report nằm ở `docs/reports/phase1d_rlaif_answer_labels_template.md`.

Full MiMo answer-label run hiện có 192 labels, 192 valid JSON, 25 ambiguous labels, 191 scored labels và correlation với RAGAS answer relevancy khoảng 0.277. Report held-out sau khi rebuild reward bằng AI-judge labels nằm ở `docs/reports/phase1d_rlaif_ai_judge_heldout_eval.md`. Đây là AI-judge feedback, không phải human labels.

4.4 Context judge labeler

Command hiện có:

```
uv run rag-bench rlaif-label-contexts \
  --actions benchmark_results/rlaif/<run-name>/rlaif_actions.jsonl \
  --output benchmark_results/rlaif/<run-name>/rlaif_context_labels_mimo.jsonl \
  --judge-provider mimo \
  --judge-model mimo-v2.5-pro \
  --resume \
  --json-retries 1 \
  --max-completion-tokens 4096
```

rlaif-label-contexts judge chỉ từ question, optional logged answer và retrieved chunks đã log. Output ghi sufficient, selected_chunk_ids, redundant_chunk_ids, irrelevant_chunk_ids, missing_evidence, minimality_score, evidence_support_score và context_quality_score. Nếu judge trả chunk id không thuộc action row, id đó bị drop và ghi vào metadata. Missing context, invalid JSON, empty completion hoặc judge error đều thành ambiguous label với score null, không thành 0.

Script scripts/summarize_rlaif_context_labels.py đọc rlaif_context_labels.jsonl và báo valid/invalid JSON, ambiguous labels, sufficiency, missing evidence, selected/redundant/irrelevant chunk counts, dropped unknown chunk-id count, context quality, evidence support và minimality statistics. Template report nằm ở docs/reports/phase1d_rlaif_context_labels_template.md.

4.5 Pairwise judge labeler

Command hiện có:

```
uv run rag-bench rlaif-label-pairs \
  --actions benchmark_results/rlaif/<run-name>/rlaif_actions.jsonl \
  --rewards benchmark_results/rlaif/<run-name>/rlaif_rewards.jsonl \
  --preferences benchmark_results/rlaif/<run-name>/rlaif_preferences.jsonl \
  --output benchmark_results/rlaif/<run-name>/rlaif_pairwise_labels_mimo.jsonl \
  --judge-provider mimo \
  --judge-model mimo-v2.5-pro \
  --limit 50 \
  --resume
```

rlaif-label-pairs dùng preference cũ chỉ để chọn cặp action có thể so sánh. Action A là action reward formula đang chọn, Action B là action bị reject, nhưng judge được yêu cầu quyết định độc lập giữa A, B, tie hoặc ambiguous dựa trên question, answer, retrieved context và token/latency/KV cost đã log. Missing action/reward, missing answer/context, invalid JSON hoặc judge error đều thành ambiguous label với confidence null, không thành score 0. Đây là dữ liệu calibration offline, chưa train reward model, DPO hoặc thay runtime policy.

Script scripts/summarize_rlaif_pairwise_labels.py đọc rlaif_pairwise_labels.jsonl và báo A/B/tie/ambiguous counts, agreement/disagreement với reward-derived preference, confidence statistics, quality-regret count, unsupported-claim risk count và judge provider/model counts. Template report nằm ở docs/reports/phase1d_rlaif_pairwise_labels_template.md.

MiMo-50 audit đầu tiên trên reward/preference set đã rebuild bằng full MiMo answer labels có kết quả:

Metric	Value
Pair labels	50
Valid JSON	50
Invalid JSON	0
Ambiguous/timeouts	2
A wins	41
B wins	7
Ties	0
Agreement over non-ambiguous decisions	0.854
Mean confidence	0.914

Các disagreement đều nằm trong cùng một query group, nơi scalar reward thích action có quality/support score cao hơn một chút, còn direct pairwise judge xem hai câu trả lời đều acceptable hoặc đều là correct abstention rồi chọn action rẻ hơn. Điều này cho thấy hướng calibration tiếp theo nên giảm ảnh hưởng của quality delta nhỏ khi direct judge đánh dấu answer quality/evidence support là tie.

Script `scripts/diagnose_rlaif_pairwise_calibration.py` join pairwise labels với reward/action rows để tìm candidate calibration pattern. Với ngưỡng candidate quality=0.10 và support=0.20, MiMo-50 diagnostics có kết quả:

Metric	Value
Valid non-ambiguous decisions	48
Small quality/support delta pairs	38
Cheaper wins when quality/support tied	35
Scalar-over-quality disagreements	5
Scalar-over-quality disagreement rate	0.104
Cheaper-win rate when quality/support tied	0.921

Đây là diagnostic analysis-only, chưa đổi công thức reward. Candidate rule nên được bật bằng flag riêng: nếu quality/support gaps nằm trong tie band và không có unsupported-claim risk khác biệt, giảm ảnh hưởng của quality/support delta nhỏ rồi để token, latency và KV efficiency quyết định mạnh hơn.

`pairwise_tie_v1` hiện đã được thêm vào `rlaif-reward` dưới dạng candidate opt-in. Với `quality_tie_threshold=0.10`, `support_tie_threshold=0.20` và `-tie-break-by-efficiency`, calibrated candidate trên AI-judge reward set tạo 1270 preferences thay vì 722 preferences, trong đó 900 preferences có reason `pairwise_tie_v1_efficiency`. Scalar reward rows giữ nguyên, nên fixed/cheapest/best-average/oracle eval metrics hiện không đổi; tác động thực tế sẽ xuất hiện ở preference-aware hoặc learned selectors.

4.6 Reward/preference builder

Command hiện có:

```
uv run rag-bench rlaif-reward \
  --actions benchmark_results/rlaif/<run-name>/rlaif_actions.jsonl \
  --feedback benchmark_results/rlaif/<run-name>/rlaif_feedback.jsonl \
  --answer-labels benchmark_results/rlaif/<run-name>/rlaif_answer_labels_mimo.jsonl \
  --output-dir benchmark_results/rlaif/<run-name> \
  --quality-weight 0.75 \
  --support-weight 0.10 \
```

```
--token-weight 0.05 \
--latency-weight 0.05 \
--kv-weight 0.05 \
--min-reward-delta 0.03 \
--max-quality-regret 0.02
```

Khi có `-answer-labels`, reward builder dùng AI-judge labels hợp lệ làm feedback chính. Nếu label invalid, ambiguous, errored hoặc thiếu quality, builder fallback về feedback gốc như RAGAS khi có và ghi rõ merge reason trong metadata. Nếu cả hai nguồn đều thiếu quality hợp lệ, reward vẫn là null, không phải 0.

Builder ghi ba artifact:

- `rlaif_rewards.jsonl`: reward row cho mỗi action, gồm reward, reward components, weights, provenance và reward mode.
- `rlaif_preferences.jsonl`: pairwise preferences cho context-policy và retrieval-context groups.
- `rlaif_reward_summary.md`: coverage, reward modes, preference types, skip reasons và weights.

Guardrail quan trọng: nếu feedback thiếu quality hoặc ambiguous, reward vẫn được ghi để audit nhưng `reward=null`; nó không bị biến thành 0. Preference chỉ được tạo giữa các action có reward thật, cùng query group, reward gap đủ lớn, và không vi phạm `max-quality-regret`. Vì vậy action rẻ hơn không được thắng nếu answer quality tụt rõ.

4.7 Offline selector baselines

Command hiện có:

```
uv run rag-bench rlaif-split \
  --rewards benchmark_results/rlaif/<run-name>/rlaif_rewards.jsonl \
  --preferences benchmark_results/rlaif/<run-name>/rlaif_preferences.jsonl \
  --output-dir benchmark_results/rlaif/<run-name>/split_seed42 \
  --train-ratio 0.8 \
  --seed 42

uv run rag-bench rlaif-train \
  --rewards benchmark_results/rlaif/<run-name>/split_seed42/train_rewards.jsonl \
  --preferences benchmark_results/rlaif/<run-name>/split_seed42/train_preferences.jsonl \
  --output benchmark_results/rlaif/<run-name>/split_seed42/rlaif_policy.json

uv run rag-bench rlaif-eval \
  --rewards benchmark_results/rlaif/<run-name>/split_seed42/eval_rewards.jsonl \
  --policy benchmark_results/rlaif/<run-name>/split_seed42/rlaif_policy.json \
  --out-md benchmark_results/rlaif/<run-name>/split_seed42/rlaif_eval_summary.md \
  --split-manifest benchmark_results/rlaif/<run-name>/split_seed42/split_manifest.json
```

`rlaif-split` chia theo `benchmark + query_id`, không chia random theo action row. Tất cả action của cùng một query luôn nằm cùng train hoặc eval split. Preference chỉ được giữ nếu cả chosen/rejected action cùng nằm trong train hoặc cùng nằm trong eval; preference cắt ngang split bị drop và được đếm trong manifest.

`rlaif-train` hiện tạo bảy baseline offline:

- `fixed`: chọn action signature có nhiều reward thật nhất, tie-break bằng reward trung bình.
- `cheapest`: chọn action có chi phí token + latency + estimated KV thấp nhất trong logged query group.
- `best-average`: chọn action signature khả dụng có reward trung bình cao nhất trong dữ liệu train.
- `family_smoothed_best_average`: chọn theo exact signature mean nếu có; nếu không có thì backoff sang retrieval-context-family mean rồi context-policy mean.
- `shrinkage_smoothed_best_average`: score từng row bằng empirical-Bayes shrinkage từ exact signature mean về retrieval-context-family mean, context-policy mean rồi global train mean.
- `linear_reward_model`: learned offline selector bằng ridge regression trên action/context/cost features, không dùng reward, quality, evidence-support labels hoặc preference outcome làm input feature.
- `smoothed_linear_selector`: learned offline selector bằng ridge regression trên action/context/cost features cộng thêm exact/family/context-policy/retriever reward means và counts được tính chỉ từ train split; eval rows không được dùng để fill aggregate features.
- `oracle-logged`: upper bound offline, chọn action có reward quan sát cao nhất trong cùng query group.

`rlaif-eval` báo cáo mean reward, mean quality, normalized token/latency/KV cost, selected action distribution, scored coverage, selection coverage và paired oracle gap. Khi truyền `-split-manifest`, summary ghi `held_out_query_eval=true`. Policy artifact luôn ghi `runtime_default_replacement=false`; nó là artifact đánh giá offline, không thay runtime default.

Held-out smoke đầu tiên dùng split seed 42 trên reward rows đã join RAGAS. Split có 27 train queries, 7 eval queries, 178 train reward rows, 14 eval reward rows, 572 train preferences và 6 eval preferences. Trên eval set, `best-average` đạt mean reward 0.425 và mean quality 0.538 với coverage 0.889; `cheapest` đạt mean reward 0.377 và mean quality 0.476 với coverage 1.0. Kết quả này củng cố trade-off chính: tiết kiệm token/KV đơn thuần làm giảm quality, còn selector tốt cần giữ quality guardrail.

Sau khi có full MiMo answer labels và candidate `pairwise_tie_v1`, `linear_reward_model` được train trên 178 train reward rows và eval trên 9 held-out query groups. Trong eval nhỏ này, `linear_reward_model` đạt coverage 1.0, mean reward 0.713, mean quality 0.872 và paired oracle gap 0.075; `cheapest` đạt reward 0.704, quality 0.861 và oracle gap 0.083; `best-average` đạt reward 0.707, quality 0.869 với coverage 0.889. Đây là sanity check cho learned offline selector, chưa phải generalization claim lớn vì eval set còn nhỏ.

Multi-seed sweep với seeds 1, 2, 3, 4, 5 và 42 cho thấy seed 42 hơi lạc quan. Trung bình 6 seed, `linear_reward_model` giữ coverage 1.0 và tốt hơn `cheapest` về reward trung bình (0.586 so với 0.577) và oracle gap (0.086 so với 0.094), nhưng chưa vượt `best-average` về reward/quality trung bình. Điều này biến kết quả v2 thành positive first signal thay vì claim ổn định.

Action coverage diagnostics cho thấy bottleneck chính là exact-signature sparsity. Có 77 exact signatures trên 192 reward rows và 34 queries; 20 signatures là singleton (26.0%). Trung bình qua 6 split, exact-signature eval group coverage là 0.911, khớp với coverage của `best-average`. Khi collapse xuống `retrieval_context_family` gồm retriever, context policy, budget bucket và adaptive profile, eval group coverage lên 1.0. Vì vậy bước kế tiếp nên là family-level smoothing/backoff, chưa phải pairwise ranker phức tạp.

`family_smoothed_best_average` đã được thêm để kiểm chứng giả thuyết này. Trung bình 6 seed, policy này đạt coverage 1.0, reward 0.598, quality 0.767 và oracle gap 0.074. Nó sửa coverage loss của `best-average` và giảm oracle gap (0.074 so với 0.080), nhưng vẫn thấp hơn `best-average` về reward/quality trung bình (0.598/0.767 so với 0.618/0.794). Kết luận: family smoothing hữu ích như coverage repair, nhưng chưa đủ để thay thế exact-signature ranking.

`smoothed_linear_selector` kiểm tra xem `aggregate train means` có giúp `learned selector` tốt hơn không. Policy này giữ coverage 1.0 và cải thiện nhẹ so với `linear_reward_model`: reward 0.595 so với 0.586, oracle gap 0.076 so với 0.086. Tuy nhiên nó vẫn thấp hơn `family_smoothed_best_average` và `best-average`. Diễn giải đúng là `aggregate train means` hữu ích, nhưng trong `small-data logged-candidate regime` hiện tại, `simple family smoothing` và `exact-signature averaging` vẫn là baseline rất mạnh.

`shrinkage_smoothed_best_average` sửa điểm yếu của `hard backoff` bằng cách score từng row thay vì dừng ở level `exact-signature` cho cả query group. Trung bình 6 seed, policy này đạt coverage 1.0, reward 0.602, quality 0.773 và oracle gap 0.070. Đây là `full-coverage non-oracle baseline` mạnh nhất hiện tại, nhưng vẫn thấp hơn `best-average` về reward/quality trung bình. Ngoài ra, `pairwise_tie_v1` hiện mới ảnh hưởng preference construction; các selector reward-based này vẫn train từ scalar reward rows. Pairwise preferences sẽ đi vào selection sau khi có pairwise ranker hoặc calibrated scalar reward path. Các cost feature hiện cũng là `logged/offline normalized costs`; runtime cần estimated token/KV costs và predicted latency trước generation.

5 Hai điểm đã harden trước reward/preference

Pasted review đã chỉ ra hai điểm yếu hợp lý cần sửa trước khi xây reward/preference builder. Hai điểm này đã được xử lý trong hardening hiện tại.

5.1 Judge provenance cho non-MiMo

Trước hardening, `_feedback_from_judge` map provider khác mimo thành `heuristic`. Điều này có thể làm DeepSeek hoặc Groq judge bị ghi nhầm là `heuristic`. Cách sửa đã chọn là:

- mở rộng provenance thành `ai_judge`;
- lưu provider cụ thể ở `judge_provider`;
- lưu model cụ thể ở `judge_model`;
- giữ `mimo_judge` trong schema để backward compatibility với artifact cũ.

Như vậy MiMo, DeepSeek, Groq hoặc judge LLM khác không bị trộn vào `heuristic`. `heuristic` chỉ còn dùng cho label không đến từ LLM judge.

5.2 Full-context/legacy action không có explicit budget

`RetrievalContextAction.budget_chars` trước đây là positive int bắt buộc. Với legacy/full action hoặc output cũ không có explicit budget, converter có thể invalid. Schema hiện cho phép:

```
budget_chars: int | None
```

Nếu `budget_chars=None`, action id vẫn ổn định và biểu diễn đúng “full-context” hoặc legacy baseline. Điều này hợp hơn với Phase 1D, vì baseline full/legacy phải có mặt trong action space thay vì bị loại vì thiếu metadata.

6 Local Qwen KV Estimate

Script `scripts/estimate_local_qwen_kv_cache.py` ước lượng KV-cache cho các model Qwen2.5 mà không load weights. Công thức dùng:

$$\text{KV bytes} = 2 * \text{layers} * \text{num_key_value_heads} * \text{head_dim} \\ * \text{seq_len} * \text{batch_size} * \text{dtype_bytes}$$

Report `docs/reports/local_qwen_kv_estimates.md` hiện ghi bảng cho Qwen2.5 0.5B, 1.5B, 3B, 7B và 14B ở 1K, 2K, 4K, 8K, 16K, 32K và 128K tokens với batch size 1, dtype 2 bytes. Ví dụ analytical KV-cache của Qwen2.5-14B là khoảng 0.750 GiB ở 4K, 6.000 GiB ở 32K và 24.000 GiB ở 128K. Đây chỉ là KV-cache estimate; chưa bao gồm weights, activations, allocator fragmentation, paged-attention block overhead hoặc runtime server overhead.

7 Roadmap tiếp theo

Thứ tự tiếp theo nên giữ như sau:

1. **Context judge run:** chạy `rlaif-label-contexts` trên subset nhỏ sau khi answer-label và pairwise audit đã ổn, rồi dùng labels này để tạo context-sufficiency preferences.
2. **Context label report:** chạy `scripts/summarize_rlaif_context_labels.py` và điền `docs/reports/phase1d_rlaif_context_labels_template.md`.
3. **More logged query groups:** tăng số query/action groups và retriever diversity trước khi kết luận selector learned có generalization ổn định.
4. **Richer pairwise sample:** chạy pairwise trên sample đa dạng hơn sau khi context labels được populate.
5. **Simple ranking baseline:** sau khi có thêm data/context labels, cân nhắc pairwise ranker nhẹ từ preference rows.
6. **Contextual bandit thật:** chỉ sau khi offline eval cho thấy không làm giảm answer quality.

Guardrail chính phải giữ: learned RLAIF/bandit policy không được thay adaptive-heuristic làm default runtime policy trước khi pass offline evaluation và quality guardrails. Web-search actions chỉ là live stress-test actions, không trộn với reproducible BEIR benchmark claims.

8 Kết luận

Repo hiện đã có engineering story tốt hơn trước: plan checkpoint, schema checkpoint, dataset builder checkpoint, reward/preference checkpoint, offline selector checkpoint và test suite pass. Bổ sung research grounding giúp báo cáo không còn giống một pipeline tự phát, mà đứng tại giao điểm của RAG evaluation, prompt/context compression, KV-cache compression, RLAIF và contextual bandit learning.

Điểm cần làm ngay tiếp theo là hoàn tất judge runs và tăng coverage dữ liệu trước khi đi sang simple ranking model hoặc contextual bandit thật. Đây là đường đi đúng để RLAIF trong project trở thành một cơ chế học action có dữ liệu và guardrail, không phải một nhãn “RL” gán sau.

Tài liệu

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Chris Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Nova DasSarma Mercado, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022. <https://arxiv.org/abs/2212.08073>.
- [2] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyi Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024. <https://arxiv.org/abs/2406.02069>.
- [3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. <https://arxiv.org/abs/2404.16130>.
- [4] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023. <https://arxiv.org/abs/2309.15217>.
- [5] Huiqiang Jiang, Qian Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmllingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023. <https://arxiv.org/abs/2310.05736>.
- [6] Huiqiang Jiang, Qian Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023. <https://arxiv.org/abs/2310.06839>.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020. <https://arxiv.org/abs/2005.11401>.
- [8] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. *arXiv preprint arXiv:1003.0146*, 2010. <https://arxiv.org/abs/1003.0146>.
- [9] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023. <https://arxiv.org/abs/2310.06201>.
- [10] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tian Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024. <https://arxiv.org/abs/2404.14469>.
- [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024. <https://arxiv.org/abs/2402.02750>.

- [12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023. <https://arxiv.org/abs/2305.18290>.
- [13] Nandan Thakur, Nils Reimers, Andreas Ruckle, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021. <https://arxiv.org/abs/2104.08663>.
- [14] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023. <https://arxiv.org/abs/2309.17453>.
- [15] Zhenyu Zhang, Ying Sheng, Tian Zhou, Tian Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023. <https://arxiv.org/abs/2306.14048>.